

TECH INNOVATORS (GROUP 17)

Learner No.	Name	Surname, Initials	Signature
1. 22303601	Lwandi	Nhlengethwa L	L.N
2. 22112914	Thandolwethu	ShwabedeTP	T.P
3. 22353176	Olwethu	Ximba O	O.X
4. 22266435	Mhlengi	Khanyile M.C	M.C
5. 22103532	Funabani	Mthembu F.	F.
6. 2433151	Amahle	Nene	M.A Nene
7. 25071422	Luthando, Nqobani	Mbokazi	M.LN
8. 25112805	Mthokozisi	Mdletshe M.S	M.Mdlets he
9.			
10.			

1. Project Overview

Brief description of the project.

The Science Community is a web-based tutoring platform being developed for Michael Vilankulu's tutoring business, which currently operates as an in-person tutoring service focused on CAPS-curriculum science subjects (Physical Sciences, Life Sciences, Mathematics, Mathematical Literacy, Accounting, and Agricultural Sciences) for Grade 10–12 learners.

The platform digitizes and centralizes the current business, which is manual, informal operation registration, scheduling, resource sharing, communication, payments, and tutor management are all currently handled in paper-based format and in person or over the phone. As the business grows and relocates, travel has become a barrier for some learners, and there is no way for parents to track their child's progress or for Michael to track tutor performance at scale.

Problem statement

The Science Community is a tutoring business run by Michael Vilankulu, who built a strong reputation teaching Learners in group sessions at a single physical location, originally in Mbombela before relocating to Cape Town. Because he prefers to teach Learners together rather than individually and does not offer house calls, Learners must travel to his location to attend, but some Learners cannot always afford or manage this travel, causing them to miss sessions with no way to catch up or attend remotely. The move to Cape Town made this worse, as a number of Learners in Mbombela were left behind with no way to continue learning from him despite still needing his teaching, showing a real need for some form of hybrid or remote access alongside in-person classes. Even for Learners who do attend, resources are inconsistent. Michael tries to share books, notes, and materials,

but this depends on a Learner being present in class to receive them, so Learners who miss a session often miss out on the resource as well, compounding the effect of a missed class. Because sessions are taught in groups, tracking each Learner's individual progress is difficult, making it hard for Michael to know who is struggling and needs more support versus who is keeping up. Because progress and attendance aren't shared anywhere, parents also have little visibility into how their child is actually doing, relying only on what Michael or the Learner tells them. Payments add further strain, as some parents are unable to pay on time or on the expected date, leaving Michael without a reliable way to track who has paid, who owes money, and when. As Michael looks to grow the business by bringing on additional tutors to meet rising demand, he also wants to maintain the same teaching quality and reputation he has built himself, meaning new tutors need to be properly vetted rather than added carelessly. Together, these challenges make it difficult for Michael to reach Learners consistently, track their progress fairly, manage payments reliably, and grow the business without compromising the quality it's known for.

Project objectives

- Centralize registration for all user types (Tutors, Learners, Parents/Guardians) into a single digital system, replacing informal sign-up
- Replace the current paper-based scheduling (WhatsApp/phone calls) with a digital scheduling system for both group and one-on-one sessions
- Reduce travel as a barrier to attendance by laying the groundwork for hybrid (in-person + remote) session delivery
- Provide learners with a central place to access tutoring resources and materials, independent of physical attendance
- Enable individual progress tracking for learners, even within a group tutoring setting
- Give parents/guardians visibility into their child's academic progress, tutor feedback, and payment history through a dedicated dashboard
- Support a formal tutor vetting and onboarding process so quality is maintained as more tutors are added
- Centralize payment tracking across both pricing models (hourly for one-on-one, monthly for group sessions)

- Support role-based access so a single person (e.g. Michael) can hold multiple roles (Admin and Tutor) without duplicate accounts
- Build a data model and system architecture that can scale from a single-tutor operation to a multi-tutor business

2. User Stories

- As Learner, I want to register an account, so that so that the Learner can apply to receive tutoring services.
 - As a Learner, I want to view all the available subjects, so that I can enroll in the subjects I need help with.
 - As a learner, I want to book an available group tutoring session, so that I can attend sessions in my subjects.
 - As a Learner, I want to request a one-on-one session, so that receive one-on-one academic assistance.
 - As a Learner, I want to view my results, so that I can track my academic progress
 - As a learner, I want to access uploaded study resources and materials, so that I can catch up even if I miss a session.
 - As a Learner, I want to receive notifications of my sessions, so that I do not miss my tutorials.
 - As a learner, I want to view my timetable/schedule, so that I know when my sessions are.
 - As a Learner, I want to provide feedback about my tutorial sessions, so that they can improve.
 - As a learner, I want to view my attendance record, so that I know which sessions I've attended or missed.
- Admin
 - As an admin, I want to filter applications by status, so that I can focus on the ones that still need my decision.

- As an admin, I want to view the full details of a single tutor applicant, so that I can assess the applicant's suitability.
- As an admin, I want to approve or reject tutor applications, so that suitable applicants are marked as hired and receive a confirmation email, while unsuitable applicants are removed from the hiring pipeline.
- As an admin, I want to create a Tutor account using the details from an approved application, so that I don't have to re-enter the applicant's information manually

•

- As an admin, I want to view a list of all tutor applications, so that I can see who has applied
- As administrator, I want to view all Learner applications, so that I can approve.
- As administrator, I want to manage tutor accounts, so that tutors can teach approved subjects
- .As an admin, I want to activate or deactivate user accounts, so that I can control who has access to the system.
- As an Admin, I want to view all payment activities, so that I can track due payments.
- As an admin, I want to manage the list of subjects and grades offered, so that the curriculum stays organized and current.
- As an Administrator, I want to allocate striving Learners in particular modules, so that they can be tutors.
- As an admin, I want to create and manage the session schedule, so that learners and tutors can participate.
- As an administrator, I want to create accommodative timetables, so that Learners and tutors can participate/attend.
- As administrator, I want to view Learner feedback, so that I can improve/review my tutors
- As an administrator, I want to send notifications to guardians, so that I can remind them of overdue payments.
- As an administrator, I want to record all Learner payment details, so that I can make financial records.

- As an administrator, I want to
 - As administrator, I want to manage all Learner accounts, so that all Learner information remains accurate.
 - As an admin, I want to generate reports on the business (sessions, payments, tutor performance), so that I can monitor the business as it scales.

- Guardian
 - As a Guardian, I want to register my account, so that I can monitor my child's tutorial activities
 - As a Guardian, I want to view my child's academic progress, so that I can monitor my child's progress.
 - As a guardian, I want to check my child's timetable, so that I can monitor their attendance
 - As a guardian, I want to view tutorial fees, so that I can make payments.
 - As a guardian, I want to choose a payment method, so that I can pay in my preferred option.
 - As guardian, I want to receive notifications, so that I know when my payments are successful are not.
 - As a guardian, I want view/Download receipts, so that I have proof of my payments
 - As a Guardian, I want to I want to view payment due dates so that I know when payments are expected.
 - As a guardian, I want to receive notifications about sessions, progress, and payments, so that I stay informed.

- Tutor Applicant
 - As a tutor applicant, I want to fill out an application form with my details, subjects, qualifications, and location preference, so that I can apply to become a tutor at TSC
 - As a tutor applicant, I want to receive an email confirming I've been hired/approved, so that I know I've been accepted and what to do next

- Tutors

- As a Tutor, I want to view all my schedules sessions, so that I can prepare for my tutorials.
- As a Tutor, I want to receive notifications of my scheduled sessions, so that I don't miss my classes.
- As a tutor, I want to record learner results, progress, and feedback, so that academic performance is tracked.
- As a tutor, I want to select the subjects and grades I teach, so that I'm matched with the right learners and sessions.
- As a tutor, I want to review and accept one-on-one tutoring requests, so that I can manage individual sessions.
- As a Tutor, I want to be able to upload study material, so that I can share it with my Learners
- As a Tutor, I want to view Learner feedback, so that I can improve/review my tutoring skills.
- As a Tutor, I want to create a platform where my Learners can interact with each other, so that they can share information and assist each other.

3. Product Backlog

Product Backlog

The product backlog contains a prioritised list of features and requirements that will be developed for our tutoring service system. The backlog is based on the requirements of the main system users, which include Learners, guardians/parents, tutors, and administrators.

Tutor Applicant

Backlog ID	User Story	Priority	Story Points
PB01	As a tutor applicant, I want to fill out an application form with my details, subjects, qualifications, and location preference, so that I can apply to become a tutor at TSC.	High	5
PB02	As an admin, I want to view a list of all tutor applications, so that I can see who has applied.	High	3

**APDP201: APPLICATIONS DEVELOPMENT PROJECT II
TECH INNOVATORS**

PB03	As an admin, I want to filter applications by status (pending/approved/rejected), so that I can focus on the ones that still need a decision.	Medium	2
PB04	As an admin, I want to view the full details of a single application, so that I can assess the applicant's suitability.	High	2
PB05	As an admin, I want to approve an application, so that the system marks the applicant as hired and triggers a confirmation email.	High	5
PB06	As an admin, I want to reject an application, so that unsuitable applicants are filtered out of the pipeline.	High	2
PB07	As a tutor applicant, I want to receive an email confirming I've been hired/approved, so that I know I've been accepted and what to do next.	High	3
PB08	As an admin, I want to create a Tutor account using the details from an approved application, so that I don't have to re-enter the applicant's information manually.	High	5

Registration and Log In

Backlog ID	User Story	Priority	Story Points
PB09	As a Learner, I want to register and log in to my account so that I can access the tutoring system.	High	3
PB10	As a guardian, I want to register an account so that I can manage my child's tutoring activities	High	3
PB11	As a tutor, I want to log in so that I can access my tutoring activities.	High	3

Subject, Grade & Learner Management

Backlog ID	User Story	Priority	Story Points
PB12	As an admin, I want to manage Learners so that Learner information is kept up to date.	High	3
PB13	As an admin, I want to manage tutors so that tutors can be assigned to Learners.	High	5
PB14	As a Learner, I want to view available subjects so that I can choose the subjects I need help with.	High	3

Scheduling & Sessions

**APDP201: APPLICATIONS DEVELOPMENT PROJECT II
TECH INNOVATORS**

Backlog ID	User Story	Priority	Story Points
PB15	As a Learner, I want to request tutoring so that I can receive help in a selected subject.	High	5
PB16	As an admin, I want to create a timetable so that tutoring sessions can be properly scheduled.	High	5
PB17	As a tutor, I want to view my timetable so that I know when my tutoring sessions take place.	High	3
PB18	As a Learner, I want to view my timetable so that I know when my tutoring sessions take place.	High	3

Payments

Backlog ID	User Story	Priority	Story Points
PB19	As a guardian, I want to make payments for tutoring services so that my child's sessions can continue.	High	5
PB20	As a guardian, I want to view payment history so that I can keep track of payments made.	Medium	3
PB21	As an admin, I want to send notifications to guardians, so that I can remind them of overdue payments.	Medium	3
PB22	As a guardian, I want to choose a payment method and view/download receipts, so that I can pay in my preferred way and have proof of payment.	Medium	3

Feedback & Progress

Backlog ID	User Story	Priority	Story Points
PB23	As a Learner, I want to provide feedback about my tutor so that I can share my experience.	Medium	3
PB24	As an admin, I want to receive and view Learner feedback so that I can monitor tutor performance.	Medium	3
PB25	As a tutor, I want to record Learner progress so that academic improvement can be monitored.	Medium	5
PB26	As a guardian, I want to view my child's progress so that I can monitor their academic performance.	Medium	5

Reporting

Backlog ID	User Story	Priority	Story Points
PB27	As an admin, I want to generate reports so that I can monitor the tutoring service.	Medium	5

Notification

Backlog ID	User Story	Priority	Story Points
PB28	As a Learner, I want to receive notifications about my sessions, so that I do not miss my tutorials.	Medium	2
PB29	As a tutor, I want to receive notifications about my scheduled sessions, so that I don't miss my classes.	Medium	2
PB30	As a guardian, I want to receive notifications about sessions, progress, and payments, so that I stay informed.	Medium	2

Priority Key

- High – Essential features required for the system to function.
- Medium – Important features that improve the system.
- Low – Additional features that can be developed later.

4. Requirements Specification

4.1 Functional Requirements

Registration & Account Management

ID	Requirement	Source
FR1	The system shall allow learners to register, capturing grade (10–12) and required subjects.	Student
FR2	The system shall allow a learner to link a parent/guardian's contact number at registration or afterward.	Student / Guardian
FR3	The system shall allow tutors to register and submit qualifications/bio for vetting.	Tutor

**APDP201: APPLICATIONS DEVELOPMENT PROJECT II
TECH INNOVATORS**

FR4	The system shall allow the admin to review and approve or reject tutor applications.	Admin
FR5	The system shall allow guardians to register and link to their child's account via phone number.	Guardian
FR6	The system shall support a single account holding both Admin and Tutor roles simultaneously.	Admin
FR7	The system shall allow the admin to activate or deactivate user accounts.	Admin
FR8	The system shall enforce unique email addresses and prevent duplicate registrations.	System

Subjects & Scheduling

ID	Requirement	Source
FR9	The system shall allow the admin to manage the list of subjects and grade levels offered.	Admin
FR10	The system shall allow tutors to select the subjects and grade levels they teach.	Tutor
FR11	The system shall allow learners to view and enroll in available subjects.	Student
FR12	The system shall allow learners to browse and book available group sessions.	Student
FR13	The system shall allow learners to submit one-on-one tutoring requests.	Student
FR14	The system shall allow tutors to accept or manage one-on-one tutoring requests.	Tutor
FR15	The system shall allow the admin to oversee and manage the group session schedule.	Admin

**APDP201: APPLICATIONS DEVELOPMENT PROJECT II
TECH INNOVATORS**

FR16	The system shall display a timetable/schedule to learners, guardians, and tutors.	Student / Guardian / Tutor
------	---	-------------------------------

Resources

ID	Requirement	Source
FR17	The system shall allow tutors to upload study material for their students.	Tutor
FR18	The system shall allow learners to access resources independent of physical attendance.	Student

Progress & Feedback

ID	Requirement	Source
FR19	The system shall allow tutors to record learner progress, results, and feedback per session.	Tutor
FR20	The system shall allow learners to view their own results and tutor feedback.	Student
FR21	The system shall allow guardians to view their child's progress and tutor comments via a dashboard.	Guardian
FR22	The system shall allow learners to submit feedback on tutorial sessions.	Student
FR23	The system shall allow the admin to view learner feedback to review tutor performance.	Admin

Payments

ID	Requirement	Source
FR24	The system shall track payment status per learner across both hourly (1-on-1) and monthly (group) pricing models.	Admin

FR25	The system shall allow guardians to view tutorial fees and payment due dates.	Guardian
FR26	The system shall allow guardians to select a payment method and pay through the platform.	Guardian
FR27	The system shall allow guardians to view and download payment receipts.	Guardian
FR28	The system shall allow the admin to view all payment activity and generate financial records.	Admin

Notifications

ID	Requirement	Source
FR29	The system shall notify learners of upcoming sessions.	Student
FR30	The system shall notify tutors of their scheduled sessions.	Tutor
FR31	The system shall notify guardians of session updates, progress, and payment status (including overdue reminders).	Guardian / Admin
FR32	The system shall deliver notifications via a user's preferred contact method.	System

4.2 Non-Functional Requirements

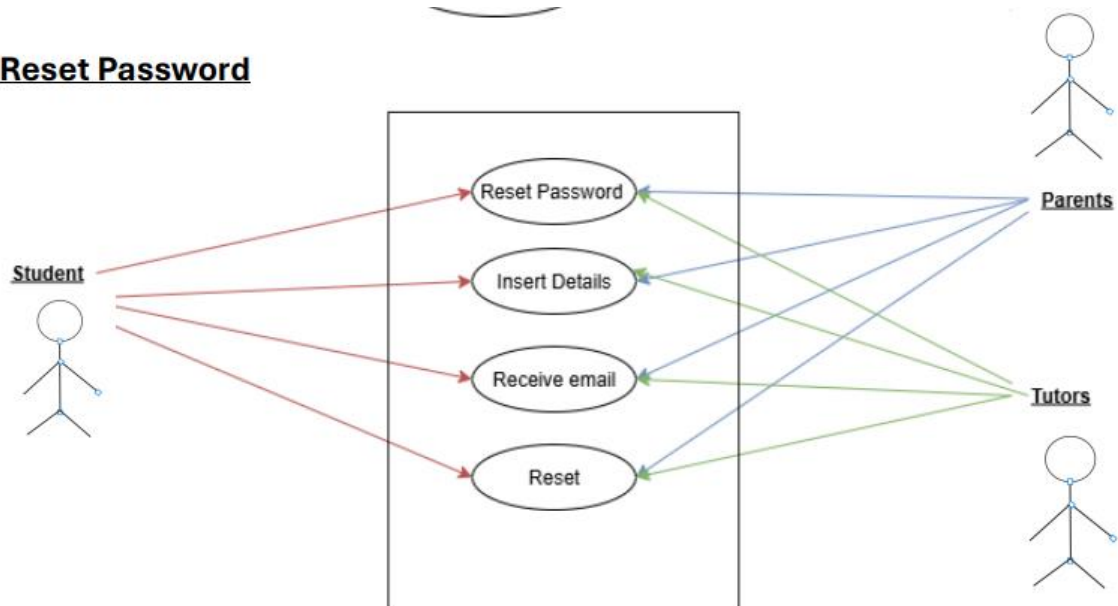
ID	Requirement	Source
NFR1	Usability	Each of the four user roles (Admin, Tutor, Learner, Guardian) shall have a dashboard tailored to that role's tasks, usable without training.
NFR2	Performance	Dashboard and schedule pages shall load within a few seconds under normal load.
NFR3	Security	The system shall enforce role-based access control at both the UI and controller/API level, not just hidden navigation.

NFR4	Security	Passwords shall be stored using industry-standard hashing (e.g. via ASP.NET Core Identity); no credentials in plain text.
NFR5	Data protection	Personal data of minors (learners) and their guardians shall be handled in line with South Africa's POPIA requirements — collect only what's needed, and restrict visibility to relevant roles.
NFR6	Reliability	Payment and attendance records shall not be lost on system failure; regular backups shall be maintained.
NFR7	Scalability	The data model and architecture shall support growth from a single tutor to a multi-tutor business without redesign.
NFR8	Maintainability	The system shall follow MVC separation of concerns (per the ASP.NET Core MVC stack) so features can be modified independently.
NFR9	Data integrity	The system shall prevent duplicate registrations and enforce referential integrity between linked parent–learner accounts.
NFR10	Availability	The system shall be accessible online for remote learners, directly addressing the travel-barrier problem identified in Section 1.

5. System Design

Class Diagram(s)

Reset Password



Use Case Diagram

Database Design (ERD)

User Interface Mock-ups/Wireframes

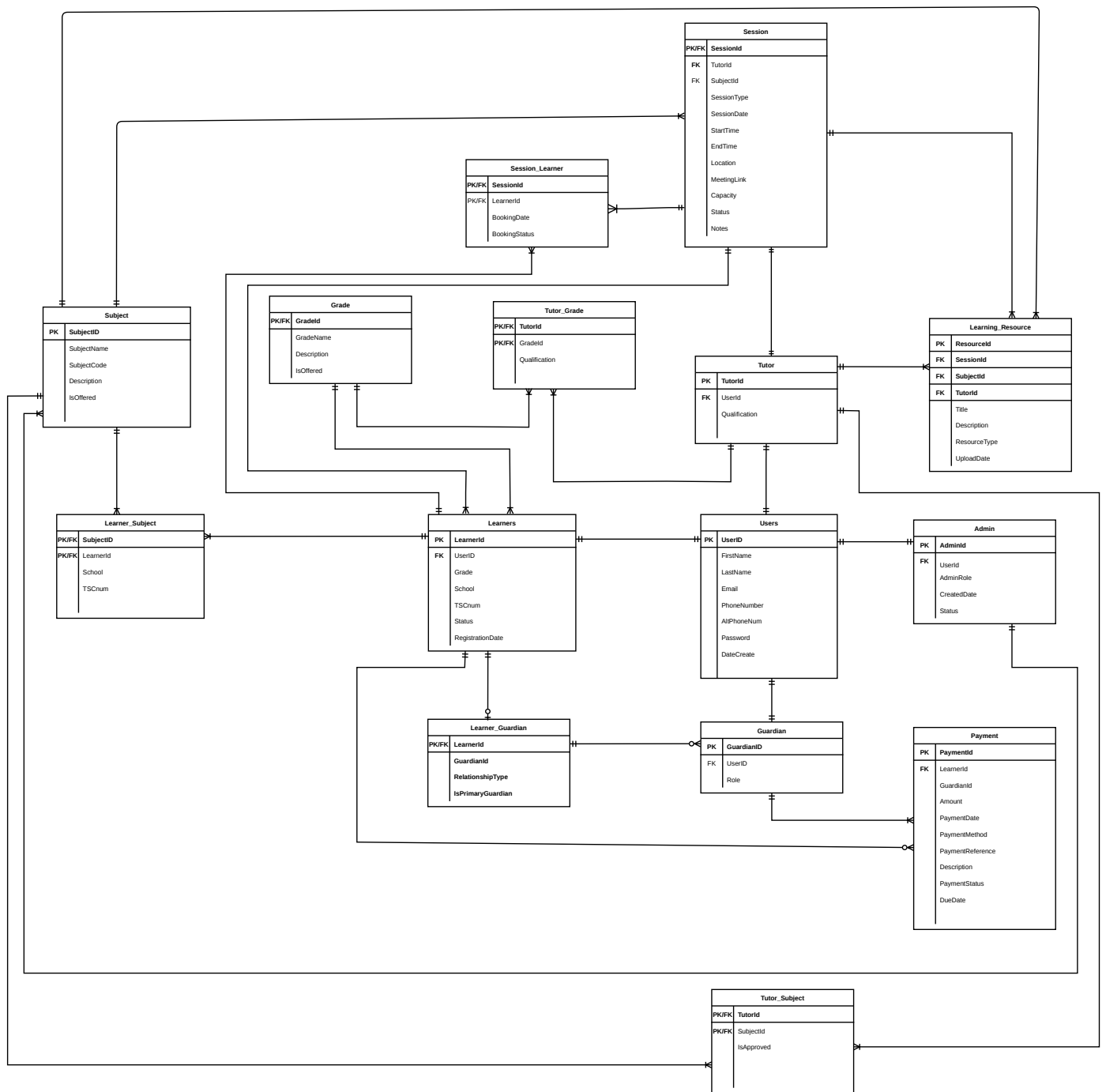
6. Sprint Backlog 1

SPRINT 1 – LAYING THE GROUNDWORK FOR THE SCIENCE COMMUNITY

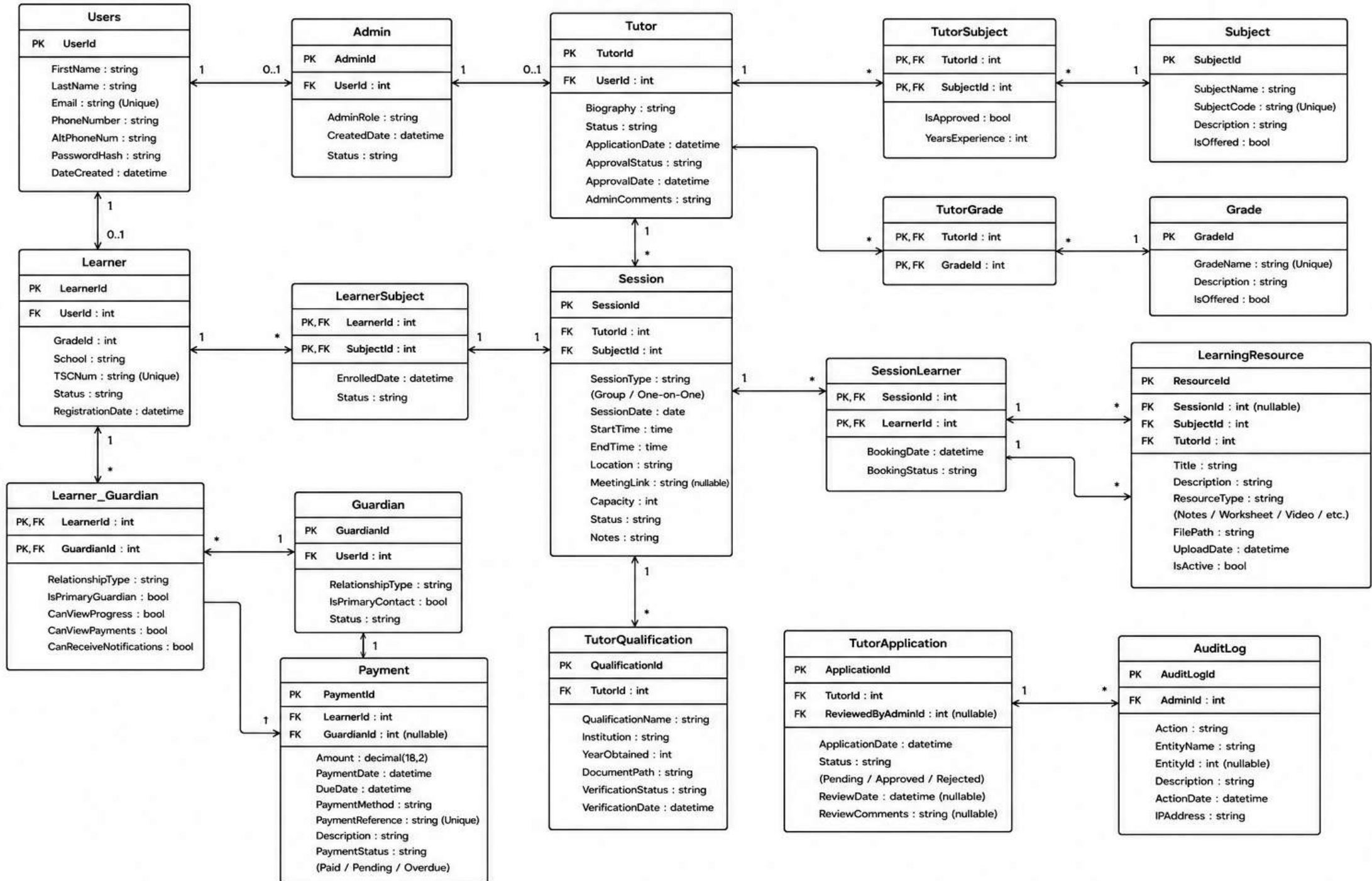
Project: Online tutoring platform for Michael Vilankulu's business

Sprint Length: 2 weeks

Big Picture Goal: Get the basics up and running so that Admins, Tutors, Learners, and Parents can all get into the system, set up their accounts, and start using the platform.



TSC CONNECT – CLASS DIAGRAM



1. WHAT WE'RE FOCUSING ON THIS SPRINT

Priority ID What We're Building

1 PB-01 A smooth registration and login experience for everyone—whether you're an Admin, Tutor, Learner, or Parent

2 PB-02 Tools for the Admin to add and organize subjects and Grades 10–12

3 PB-03 A simple process for Tutors to apply and for Admins to review and approve them

4 PB-04 A way for Parents/Guardians to connect their account to a Learner's profile

5 PB-05 Smart access controls so each person sees only what they're supposed to—and some users can wear multiple hats (like being both Admin and Tutor)

PB-01: Getting People Signed Up and Logged In

Task Time Who's Doing It

Set up the database tables for users, roles, and who's who 3h Backend Dev

Build friendly registration forms for each type of user 5h Frontend Dev

Make login/logout work smoothly, with passwords safely encrypted 4h Backend Dev

Let Parents link to their child's account using just a phone number during signup 2h Backend Dev

PB-02: Giving Admins Control Over Subjects and Grades

Task Time Who's Doing It

Create database tables for Subjects and Grades 2h Backend Dev

Build an Admin panel to add, edit, or remove subjects 4h Frontend Dev

Let Admins assign subjects to Grades 10, 11, and 12 2h Frontend Dev

Quick test to confirm Admins can manage subjects without issues 1h QA

PB-04: Connecting Parents to Learners

Task Time Who's Doing It

Set up a table in the database to link Parents and Learners 1h Backend Dev

Build a simple screen where Parents can request a link using a learner's phone number 2h Frontend Dev

Add logic to prevent duplicate learner accounts from being created 2h Backend Dev

PB-05: Role-Based Access and Multi-Role Flexibility

Task Time Who's Doing It

Integrate ASP.NET Core Identity's role system into our app 3h Backend Dev

Allow one person to hold multiple roles—e.g., Michael can be both Admin and Tutor 3h Backend Dev

Restrict pages so only the right people can access them 3h Backend Dev

PB-06: Validation, Error Handling and User Feedback

The system Validate user input during registration, Login, Tutor Applications, and parents Learner linking. Prevent duplicate user accounts and duplicate learner profiles. Display confirmation messages when registration, Applications, or account linking are completely

successfully. Handle Database and System errors, test the validation and error handling functions using both valid and invalid inputs

PB-07: Database Integration and Data Management

The system securely store, manage, and retrieve information using the SQL Server database. Create and keep the required database tables and relationships for users, roles, subjects, Tutor applications, and Parent Learner links. Implement basic operations where needed, such as adding, updating, viewing, and removing subjects. Ensure that data stored accurately and retrieved correctly. Protect user and system data from unauthorised access.

HOW WE'LL KNOW WE'RE DONE

Every user type can sign up and log in without a hitch

Admins can easily add and organize CAPS subjects for Grades 10–12

Tutors can apply, and Admins can approve or reject their applications

Parents can link to a learner and view basic info about them

The system respects roles—and someone like Michael can be both Admin and Tutor

All our code is safely pushed to GitHub with clear, meaningful commit messages

We've written and passed basic tests to confirm everything works as expected

THE TOOLS WE'LL USE

- Framework: ASP.NET Core MVC – our solid foundation
- Database: SQL Server – keeping our data organised
- Authentication: ASP.NET Core Identity – handling logins and roles securely
- Frontend: Razor Views with Bootstrap – for clean, responsive, and user-friendly pages
- Bootstrap 5 — used for all UI styling and layout (forms, cards, badges, progress indicators),

TEAM NOTES & REMINDERS

- Daily stand-ups at 9:00 AM (15 mins)
- Code reviews before merging any pull requests
- Reach out early if you're stuck—we're all in this together!
- Keep commit messages clear: e.g., “Added registration form for Tutors” not “Fixed stuff”
- Let's aim to have everything wrapped up by end of Day 9, leaving Day 10 for final testing and polish

Prepared by: the Science Community team

Date: 14 August 2026

7. Implementation

7.1 Overview

During Sprint 1, the team focused on implementing the core functionality of the tutoring system. The Sprint 1 implementation provides the foundation for learners and tutors to interact with the system and shows the main application architecture

- User registration and login.
- User authentication and authorisation.
- Student, guardian, admin, and tutor profiles.
- Tutor browsing/search functionality.
- Tutor availability management.
- Lesson/session booking.
- Dashboard functionality.
- Basic database operations.

- Admin assigning tutors
- Guardian managing payments

7.2 Programming Languages

The tutoring system was developed using the following programming languages:

Language	Purpose
[JavaScript]	Used to structure the application's web pages and user interface.
[HTML]	Used to structure the application's web pages and user interface.
[CSS]	Used to style the application and provide a consistent user interface.
[SQL]	Used for database queries and management, where applicable.
C#	Used to implement the application's server-side logic, including controllers, models, view models, services, and business-rule validation. This is the primary language of the system, built on the ASP.NET Core MVC framework.
Razor (C# + HTML)	Used to build the application's dynamic views, combining server-side C# logic with HTML markup to render forms, data summaries, and admin screens.

User Registration Page

Create your account

Join as a learner, tutor, or parent

Full Name

Email Address

Password

ConfirmPassword

Your Role

① PARENT INFORMATION

Phone Number

User Dashboards

TS

TheScienceCommunity

LEARNER

Dashboard

My Subjects

Browse Sessions

Timetable

My Bookings

Payments

My Progress

My Profile

Logout

Learner Dashboard

Upcoming **Mathematics** lesson with **MephoT** is on Monday 31 Aug at 11:00 AM.

Payment of R1500.00 for Life Science was received. Thank you!

Dismiss

Quick Actions

Timetable

My Bookings

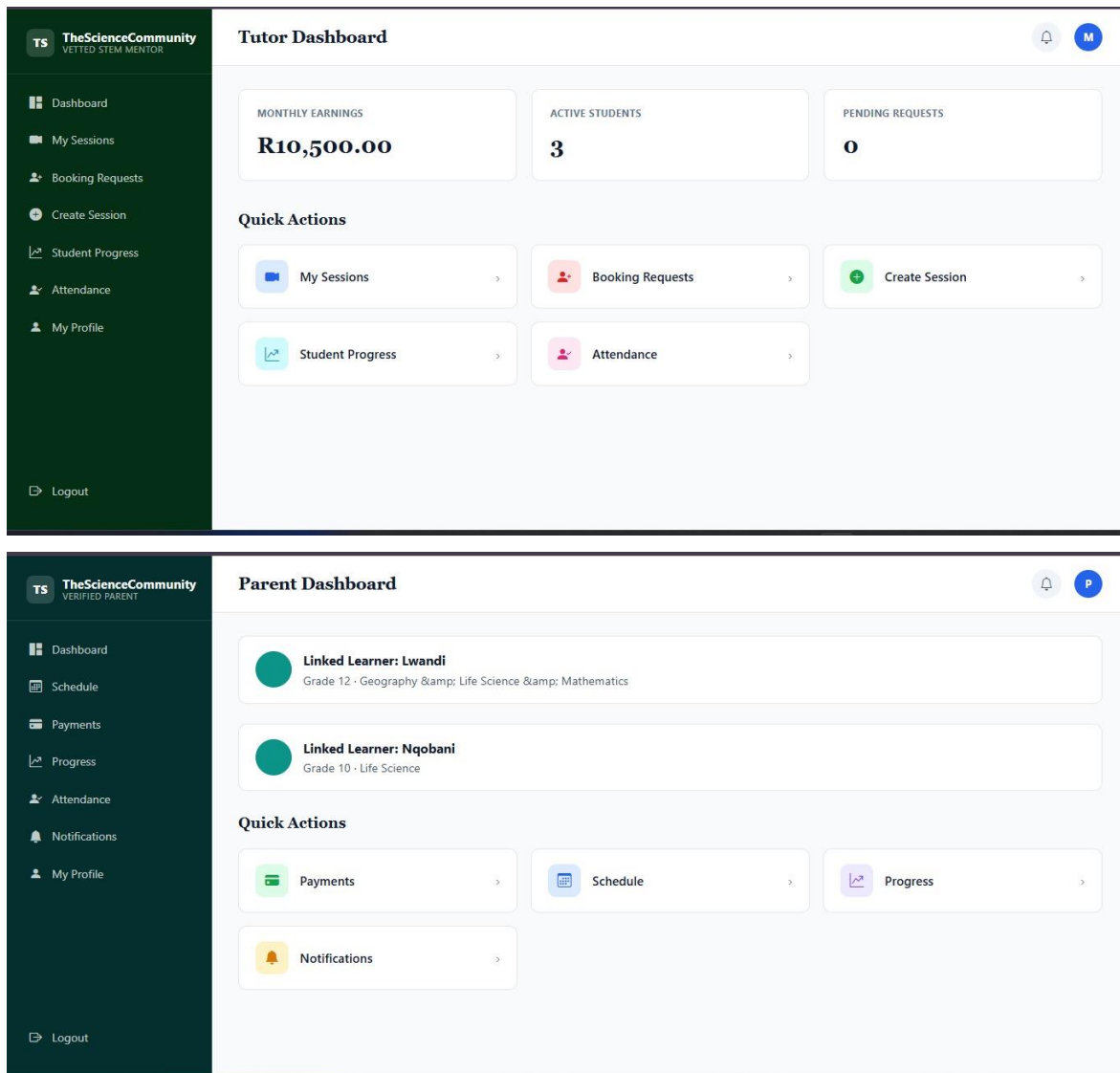
Payments

My Progress

Browse Sessions

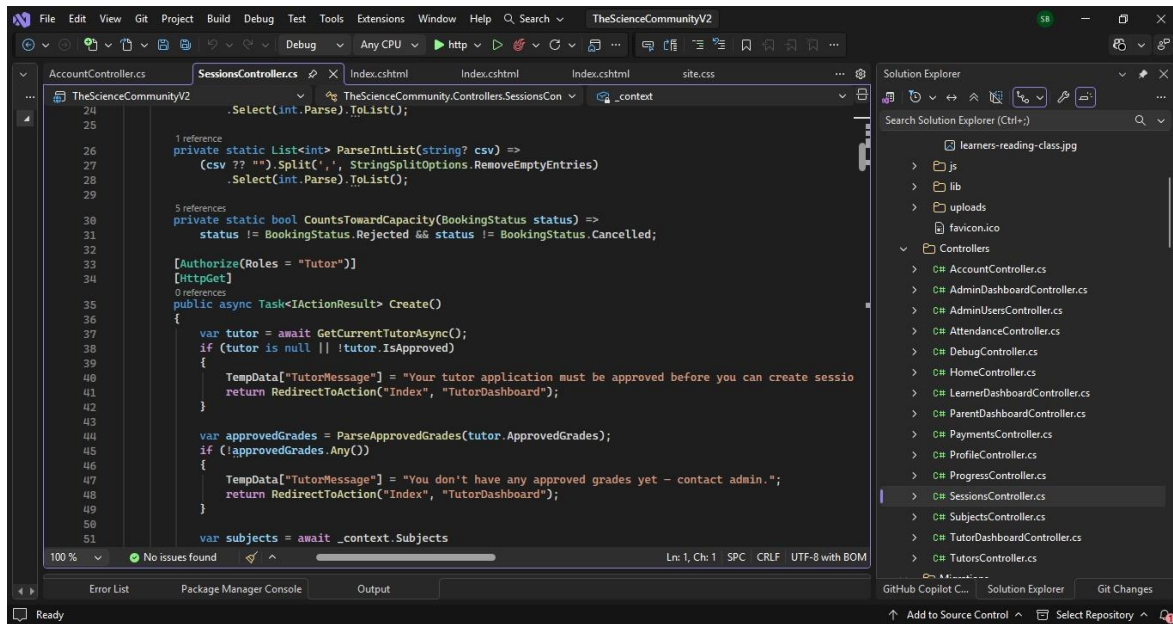
Page 21 of 30

APDP201: APPLICATIONS DEVELOPMENT PROJECT II TECH INNOVATORS



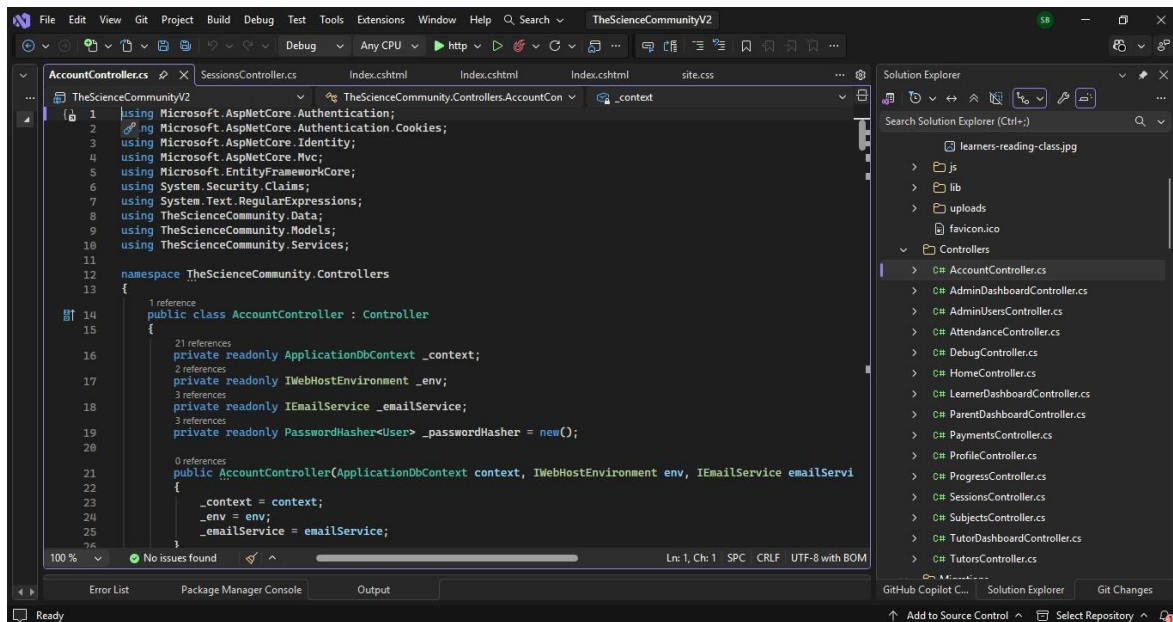
THE CODE

Session Creation



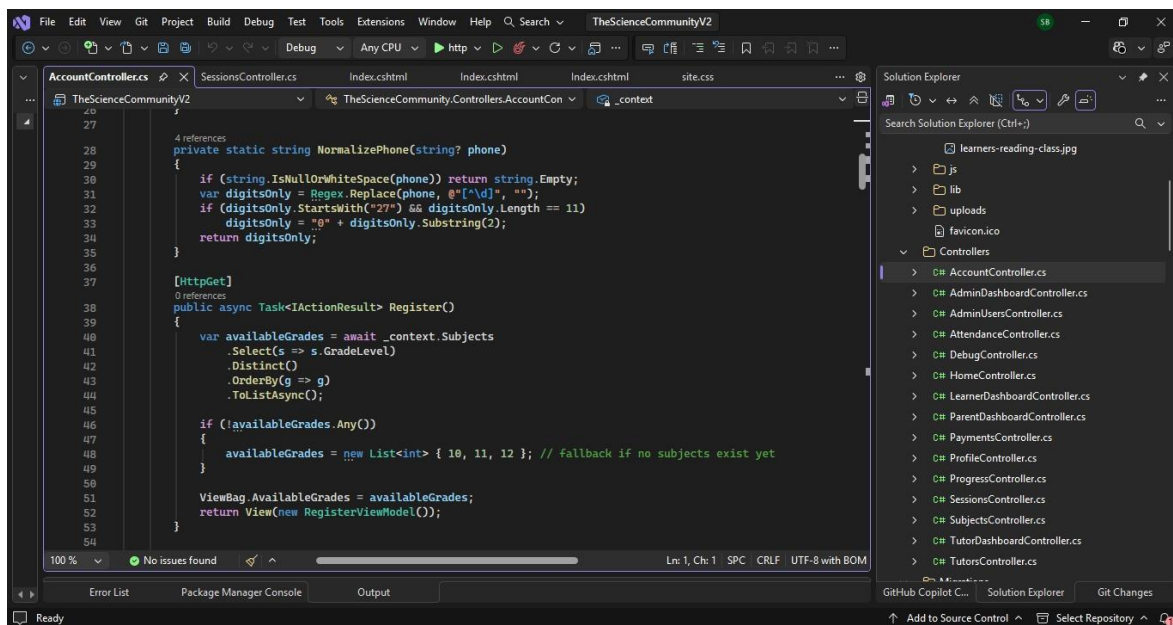
Account creation

APDP201: APPLICATIONS DEVELOPMENT PROJECT II TECH INNOVATORS



This screenshot shows the Visual Studio IDE with the `AccountController.cs` file open. The code defines the `AccountController` class, which inherits from `Controller`. It includes several private fields for dependencies like `ApplicationDbContext`, `IWebHostEnvironment`, `IEmailService`, and `PasswordHasher`. The constructor initializes these fields. The `Register` method is partially visible at the bottom of the file.

```
1 using Microsoft.AspNetCore.Authentication;
2 using Microsoft.AspNetCore.Authentication.Cookies;
3 using Microsoft.AspNetCore.Identity;
4 using Microsoft.AspNetCore.Mvc;
5 using Microsoft.EntityFrameworkCore;
6 using System.Security.Claims;
7 using System.Text.RegularExpressions;
8 using TheScienceCommunity.Data;
9 using TheScienceCommunity.Models;
10 using TheScienceCommunity.Services;
11
12 namespace TheScienceCommunity.Controllers
13 {
14     1 reference
15     public class AccountController : Controller
16     {
17         21 references
18         private readonly ApplicationDbContext _context;
19         2 references
20         private readonly IWebHostEnvironment _env;
21         3 references
22         private readonly IEmailService _emailService;
23         3 references
24         private readonly PasswordHasher<User> _passwordHasher = new();
25
26         0 references
27         public AccountController(ApplicationDbContext context, IWebHostEnvironment env, IEmailService emailService)
28         {
29             _context = context;
30             _env = env;
31             _emailService = emailService;
32         }
33     }
34 }
```

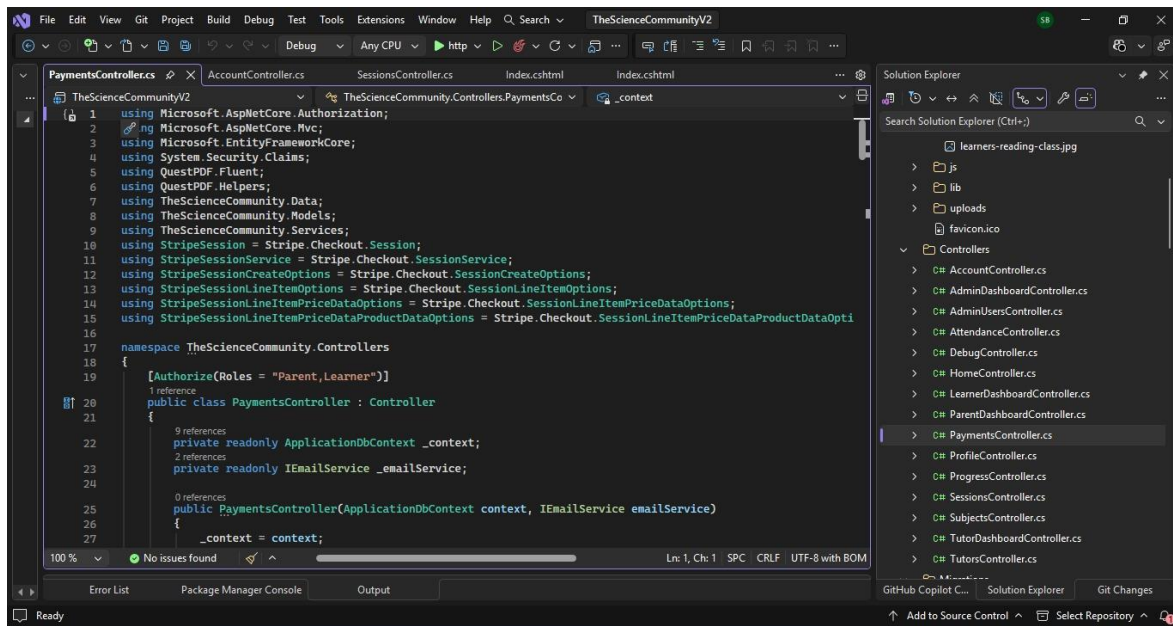


This screenshot shows the `Register` method in the `AccountController` class. The method is an asynchronous action that returns an `ActionResult`. It first retrieves a list of available grades from the database, ordered by grade level. If no grades are found, it sets a fallback list of { 10, 11, 12 }. Finally, it creates a `RegisterViewModel` and returns the view.

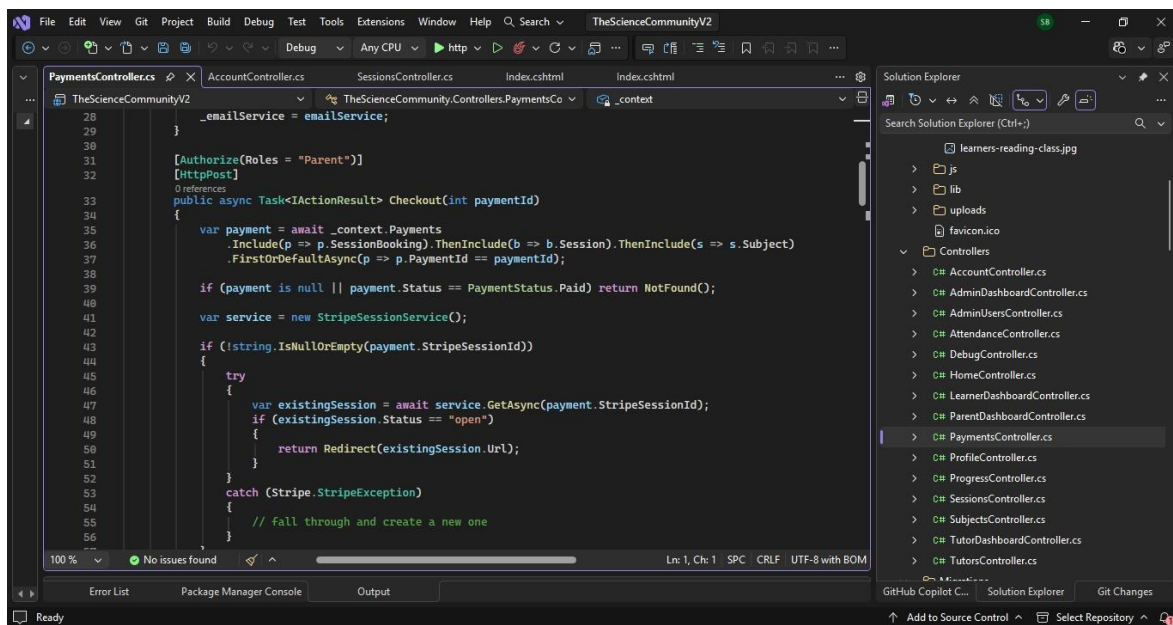
```
28     4 references
29     private static string NormalizePhone(string? phone)
30     {
31         if (string.IsNullOrWhiteSpace(phone)) return string.Empty;
32         var digitsOnly = Regex.Replace(phone, @"[^0-9]", "");
33         if (digitsOnly.StartsWith("27") && digitsOnly.Length == 11)
34             digitsOnly = "0" + digitsOnly.Substring(2);
35         return digitsOnly;
36     }
37
38     [HttpGet]
39     0 references
40     public async Task<ActionResult> Register()
41     {
42         var availableGrades = await _context.Subjects
43             .Select(s => s.GradeLevel)
44             .Distinct()
45             .OrderBy(g => g)
46             .ToListAsync();
47
48         if (!availableGrades.Any())
49         {
50             availableGrades = new List<int> { 10, 11, 12 }; // fallback if no subjects exist yet
51         }
52
53         ViewBag.AvailableGrades = availableGrades;
54         return View(new RegisterViewModel());
55     }
56 }
```

Payment Controller

APDP201: APPLICATIONS DEVELOPMENT PROJECT II TECH INNOVATORS



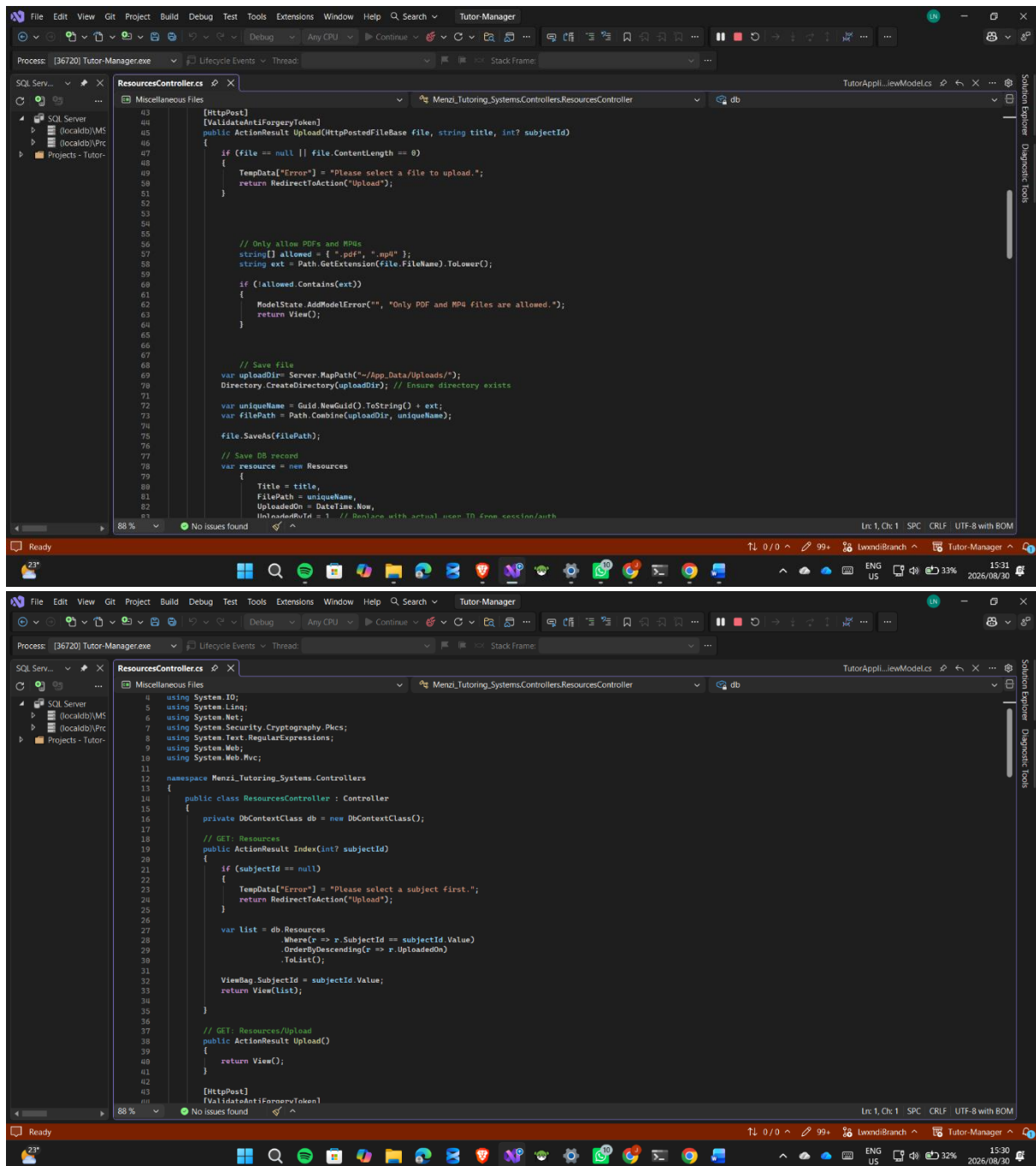
```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using Microsoft.EntityFrameworkCore;
4 using System.Security.Claims;
5 using QuestPDF.Fluent;
6 using QuestPDF.Helpers;
7 using TheScienceCommunity.Data;
8 using TheScienceCommunity.Models;
9 using TheScienceCommunity.Services;
10 using StripeSession = Stripe.Checkout.Session;
11 using StripeSessionService = Stripe.Checkout.SessionService;
12 using StripeSessionCreateOptions = Stripe.Checkout.SessionCreateOptions;
13 using StripeSessionLineItemOptions = Stripe.Checkout.SessionLineItemOptions;
14 using StripeSessionLineItemPriceDataOptions = Stripe.Checkout.SessionLineItemPriceDataOptions;
15 using StripeSessionLineItemPriceDataProductDataOptions = Stripe.Checkout.SessionLineItemPriceDataProductDataOptions;
16
17 namespace TheScienceCommunity.Controllers
18 {
19     [Authorize(Roles = "Parent,Learner")]
20     public class PaymentsController : Controller
21     {
22         private readonly ApplicationDbContext _context;
23         private readonly IEmailService _emailService;
24
25         public PaymentsController(ApplicationDbContext context, IEmailService emailService)
26         {
27             _context = context;
```



```
28     }
29     _emailService = emailService;
30
31     [Authorize(Roles = "Parent")]
32     [HttpPost]
33     public async Task<IActionResult> Checkout(int paymentId)
34     {
35         var payment = await _context.Payments
36             .Include(p => p.SessionBooking).ThenInclude(b => b.Session).ThenInclude(s => s.Subject)
37             .FirstOrDefaultAsync(p => p.PaymentId == paymentId);
38
39         if (payment is null || payment.Status == PaymentStatus.Paid) return NotFound();
40
41         var service = new StripeSessionService();
42
43         if (!string.IsNullOrEmpty(payment.StripeSessionId))
44         {
45             try
46             {
47                 var existingSession = await service.GetAsync(payment.StripeSessionId);
48                 if (existingSession.Status == "open")
49                 {
50                     return Redirect(existingSession.Url);
51                 }
52             }
53             catch (Stripe.StripeException)
54             {
55                 // fall through and create a new one
56             }
57         }
```

Uploading Resources

APDP201: APPLICATIONS DEVELOPMENT PROJECT II TECH INNOVATORS



8. GitHub Repository

APDP201: APPLICATIONS DEVELOPMENT PROJECT II TECH INNOVATORS

The screenshot shows the GitHub repository page for `github.com/Lwond/TSC-Tutor`. The main content is the README, which describes the TSC-Tutor project. The README includes a title, a description, a subtitle, and several sections explaining the project's purpose and challenges. On the right side, there are sections for 'Languages' (showing a bar chart for C#, HTML, CSS, and JavaScript) and 'Suggested workflows' (listing workflows like 'Jekyll using Docker image', '.NET Desktop', and '.NET'). The bottom of the page shows the Windows taskbar with various application icons and the system clock.

README

TSC-Tutor

A web-based tutoring platform for The Science Community, connecting tutors, learners, and parents across multiple locations (Cape Town & Mbombela). Built with ASP.NET Core MVC, SQL Server, and Entity Framework Core to handle session scheduling, missed lesson tracking, and progress visibility as the business scales.

TSC-Tutor

Web-based tutoring platform for **The Science Community**, a tutoring business run by Michael Vilankulu. Built for Grade 10–12 learners following the CAPS curriculum in South Africa.

Why This Exists

Running a tutoring business day to day comes with a few real, recurring problems.

Sessions happen across different locations. Michael's learners aren't all in one place — some are based in Cape Town, others in Mbombela. That means sessions can't just be treated as one-size-fits-all in-person meetings; some have to happen remotely, while others are in person. Without a system that accounts for this properly, there's no clean way to keep track of which sessions are online, which are in person, and how each one actually needs to run.

Learners sometimes miss sessions. Life happens — a learner might not be available for a scheduled session and misses it. Right now, there's no simple way to flag that a session was missed, so it can be followed up on, rescheduled, or at least recorded so nothing falls through the cracks.

Progress needs to be visible to everyone involved, not just Michael. Parents want to know how their child is doing. Tutors need to see how their own learners are progressing across sessions. Michael needs a bigger-picture view across

Languages

C# 44.8% HTML 41.6% CSS 10.2% JavaScript 3.4%

Suggested workflows

Based on your tech stack

- Jekyll using Docker image** [Configure](#)
Package a Jekyll site using the jekyll/builder Docker image.
By GitHub Actions
- .NET Desktop** [Configure](#)
Build, test, sign and publish a desktop application built on .NET.
By GitHub Actions
- .NET** [Configure](#)
Build and test a .NET or ASP.NET Core project.
By GitHub Actions

[More workflows](#)

The screenshot shows the GitHub repository page for `github.com/Lwond/TSC-Tutor`. The main content is the README, which describes the TSC-Tutor project. The README includes a title, a description, a subtitle, and several sections explaining the project's purpose and challenges. On the right side, there are sections for 'Languages' (showing a bar chart for C#, HTML, CSS, and JavaScript) and 'Suggested workflows' (listing workflows like 'Jekyll using Docker image', '.NET Desktop', and '.NET'). The bottom of the page shows the Windows taskbar with various application icons and the system clock.

README

TSC-Tutor

A web-based tutoring platform for The Science Community, connecting tutors, learners, and parents across multiple locations (Cape Town & Mbombela). Built with ASP.NET Core MVC, SQL Server, and Entity Framework Core to handle session scheduling, missed lesson tracking, and progress visibility as the business scales.

TSC-Tutor

Web-based tutoring platform for **The Science Community**, a tutoring business run by Michael Vilankulu. Built for Grade 10–12 learners following the CAPS curriculum in South Africa.

Why This Exists

Running a tutoring business day to day comes with a few real, recurring problems.

Sessions happen across different locations. Michael's learners aren't all in one place — some are based in Cape Town,

Files

File	Description	Last Commit
README.md	Enhance README with project overview and user types	last week
Tutor-Manager.csproj	Initial commit - Tutor Manager base project	last week
Tutor-Manager.slnx	Initial commit - Tutor Manager base project	last week
appsettings.Development.json	Initial commit - Tutor Manager base project	last week
appsettings.json	Initial commit - Tutor Manager base project	last week

README

TSC-Tutor

A web-based tutoring platform for The Science Community, connecting tutors, learners, and parents across multiple locations (Cape Town & Mbombela). Built with ASP.NET Core MVC, SQL Server, and Entity Framework Core to handle session scheduling, missed lesson tracking, and progress visibility as the business scales.

TSC-Tutor

Web-based tutoring platform for **The Science Community**, a tutoring business run by Michael Vilankulu. Built for Grade 10–12 learners following the CAPS curriculum in South Africa.

Why This Exists

Running a tutoring business day to day comes with a few real, recurring problems.

Sessions happen across different locations. Michael's learners aren't all in one place — some are based in Cape Town,

Contributors

1 contributor: [Lwond](#) (Lwandi Nhlengethwa)

Languages

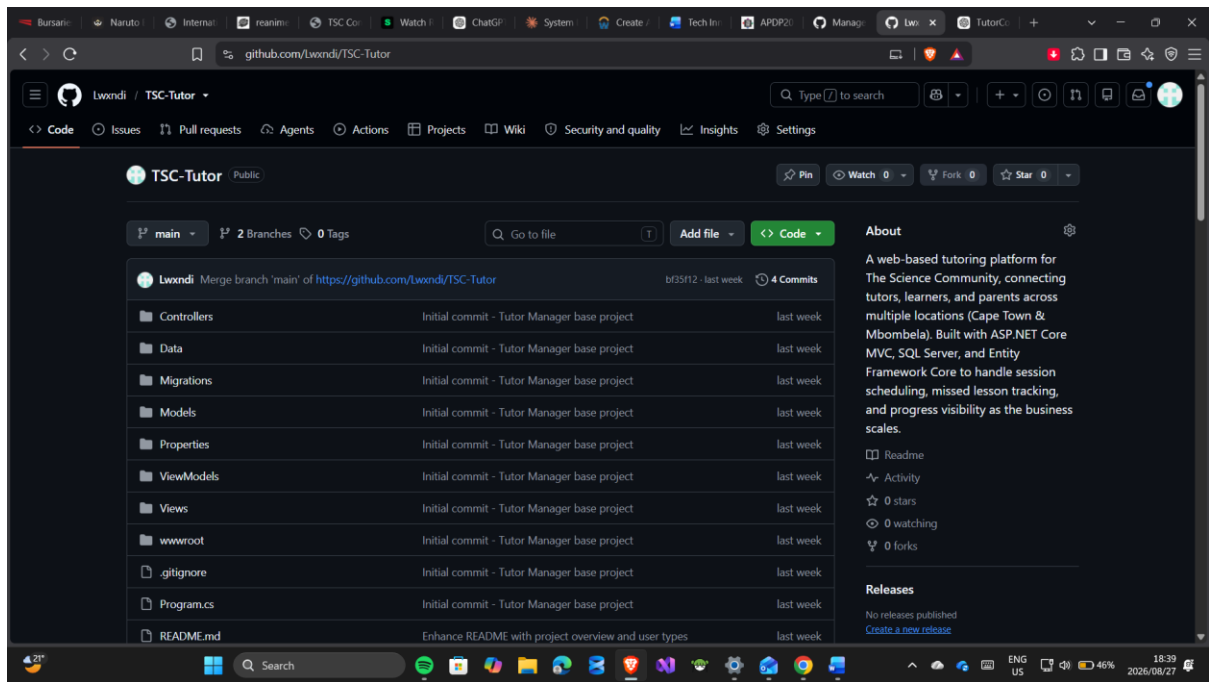
C# 44.8% HTML 41.6% CSS 10.2% JavaScript 3.4%

Suggested workflows

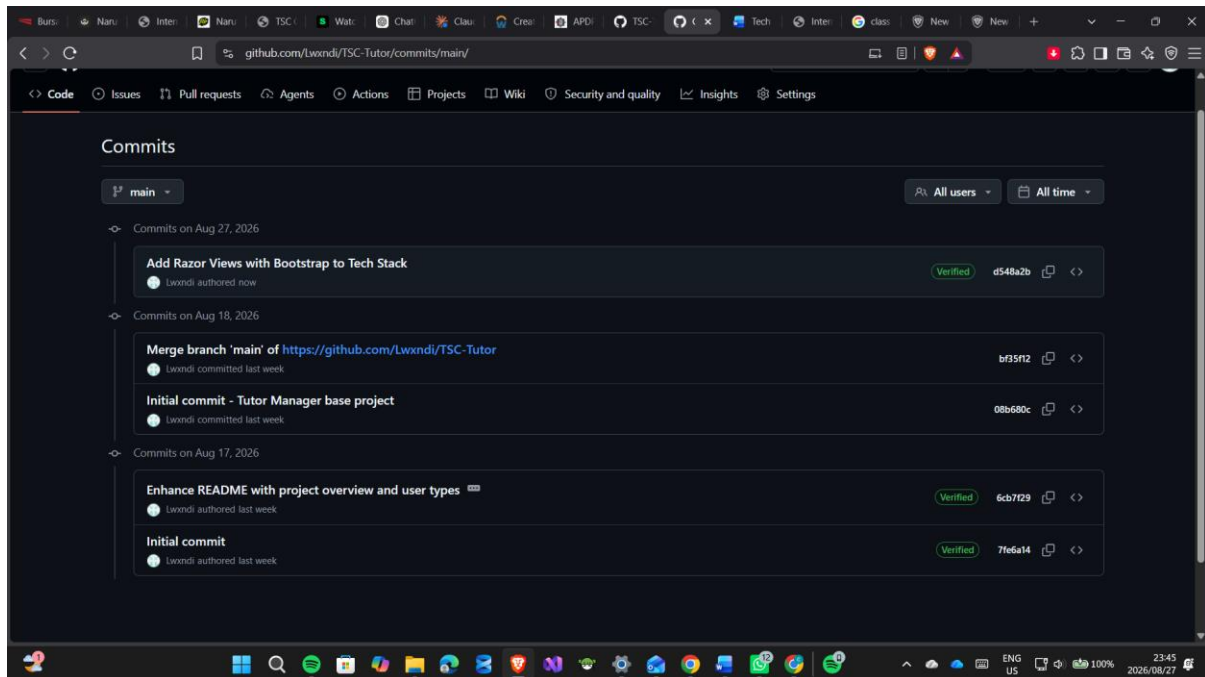
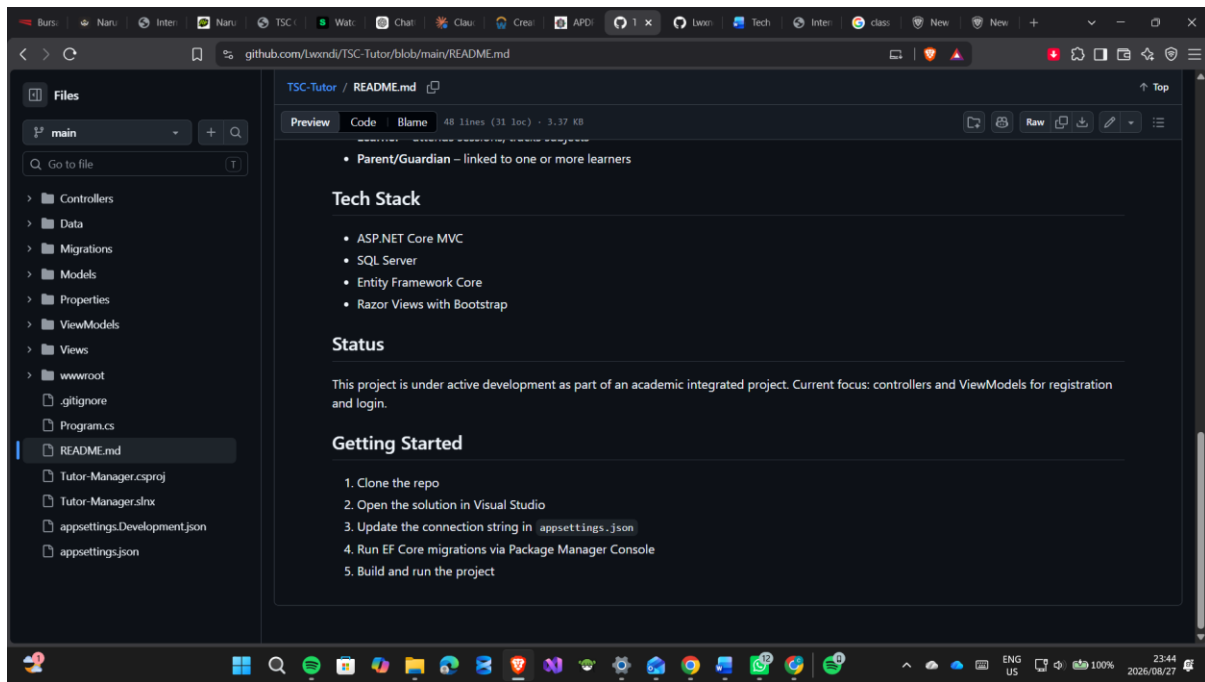
Based on your tech stack

- Jekyll using Docker image** [Configure](#)
Package a Jekyll site using the jekyll/builder Docker image.
By GitHub Actions
- .NET Desktop** [Configure](#)
Build, test, sign and publish a desktop application built on .NET.
By GitHub Actions

APDP201: APPLICATIONS DEVELOPMENT PROJECT II TECH INNOVATORS



APDP201: APPLICATIONS DEVELOPMENT PROJECT II TECH INNOVATORS



The GitHub repository URL: <https://github.com/Lwxndi/TSC-Tutor>

